

## **Implementación de control de versiones y automatización (DevOps) U3**

Juan Sebastian Cuervo Baquero

Luis Eduardo Gonzalez Mejia

Hermes Alexander Patichoy Calpa

Universidad La Sabana

Maestría Ingeniería de Software

Gestión de Modelos de Desarrollo de Software

2026

## Tabla de Contenido

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| <b>1. Resumen ejecutivo.....</b>                                     | <b>3</b>  |
| <b>2. Comparación: sistema anterior vs. sistema actual.....</b>      | <b>3</b>  |
| <b>3. Mejoras implementadas.....</b>                                 | <b>4</b>  |
| 3.1 Gestión de código y calidad.....                                 | 4         |
| 3.2 Migración y gestión de bases de datos.....                       | 4         |
| 3.3 Mejoras de arquitectura y conectividad.....                      | 5         |
| 3.4 Contenerización y registro de imágenes.....                      | 5         |
| 3.5 Automatización CI/CD.....                                        | 5         |
| 3.6 Ajustes de estabilidad detectados durante la automatización..... | 6         |
| 3.7 Validación funcional en producción.....                          | 6         |
| <b>4. Estado frente a la rúbrica de la actividad.....</b>            | <b>7</b>  |
| <b>5. Qué falta para cerrar la rúbrica.....</b>                      | <b>8</b>  |
| 5.1 Smoke tests post-deploy.....                                     | 8         |
| 5.2 Versionamiento semántico.....                                    | 8         |
| 5.3 Modelo de gestión de configuración.....                          | 8         |
| 5.4 Documento técnico README.md de 5–7 páginas.....                  | 9         |
| <b>6. Ayuda documentación.....</b>                                   | <b>10</b> |
| 6.1 Línea de tiempo técnica que debe quedar documentada.....         | 10        |
| 6.2 Evidencia de migración de base de datos y configuración.....     | 11        |
| 6.3 Evidencia del pipeline y automatización.....                     | 11        |
| 6.4 Evidencia funcional y de negocio.....                            | 11        |
| 6.5 Evidencia de resolución de incidentes y mejora del código.....   | 12        |
| 6.6 Matriz de evidencias recomendada para el README.....             | 12        |
| 6.7 Información vital que debe quedar escrita.....                   | 13        |
| 6.8 Estructura sugerida de la carpeta de documentación.....          | 13        |
| <b>7. Conclusiones.....</b>                                          | <b>14</b> |
| <b>Referencias.....</b>                                              | <b>15</b> |

## 1. Resumen ejecutivo

El proyecto evolucionó desde una solución orientada principalmente a ejecución local hacia una arquitectura contenedorizada y desplegada en nube. La nueva solución integra pruebas automatizadas, construcción de imágenes Docker, publicación en GitHub Container Registry (GHCN), despliegue en Render mediante Deploy Hooks y una base de datos PostgreSQL administrada. También se realizaron ajustes de conectividad, CORS, variables de entorno, rutas del API Gateway, comportamiento HTTP del frontend y estabilidad del CatalogService en contenedor.

La validación funcional alcanzada demuestra un flujo completo de negocio: autenticación, consulta de catálogo, carrito, orden, pago y generación/descarga de factura PDF. El siguiente nivel de madurez consiste en hacer que el pipeline valide automáticamente el estado real de producción después de cada despliegue, formalizar SemVer y cerrar la documentación exigida por la actividad.

## 2. Comparación: sistema anterior vs. sistema actual

| Área          | Situación anterior                                                            | Situación actual                                                                   | Impacto |
|---------------|-------------------------------------------------------------------------------|------------------------------------------------------------------------------------|---------|
| Ejecución     | Dependencia fuerte de localhost y configuración local.                        | Frontend, gateway, microservicios y PostgreSQL desplegados en Render.              | Alta    |
| Base de datos | Persistencia local y cadenas de conexión orientadas al entorno de desarrollo. | PostgreSQL administrado en Render; conexión mediante DATABASE_URL y Npgsql.        | Crítica |
| Configuración | URLs y puertos locales embebidos o dependientes del entorno.                  | Variables de entorno para servicios, puerto, base de datos y URL de producción.    | Alta    |
| Frontend      | VITE_API_URL orientada a localhost.                                           | Build de producción apunta al API Gateway de Render.                               | Alta    |
| API Gateway   | Rutas internas/localhost y CORS limitado al desarrollo.                       | URLs de seis servicios configuradas por entorno y CORS para el frontend de Render. | Crítica |

|                |                                                         |                                                                                            |         |
|----------------|---------------------------------------------------------|--------------------------------------------------------------------------------------------|---------|
| Pruebas        | Validación principalmente manual/local.                 | Pruebas backend y frontend integradas en GitHub Actions; frontend validado con 27 pruebas. | Alta    |
| Docker         | Construcción local/manual.                              | Ocho imágenes construidas automáticamente y publicadas en GHCR.                            | Crítica |
| Trazabilidad   | Sin vínculo completo entre commit e imagen desplegable. | Tags latest + SHA de GitHub para las imágenes.                                             | Alta    |
| Despliegue     | Manual por servicio en Render.                          | Deploy Hooks disparados desde GitHub Actions para los ocho servicios.                      | Crítica |
| Validación E2E | No demostrada en nube.                                  | Flujo de compra completo con pago y factura PDF en producción.                             | Crítica |

### 3. Mejoras implementadas

#### 3.1 Gestión de código y calidad

- Uso de una rama específica de trabajo (laboratorio/dockerizacion) para aislar los cambios de dockerización y automatización.
- Commits descriptivos orientados a intención (fix:, ci:, test:), mejorando trazabilidad y revisión.
- Limpieza del repositorio para evitar versionar artefactos de compilación como bin/ y obj/.
- Validación automatizada de backend .NET y frontend React antes de construir imágenes de producción.
- Ajuste de pruebas del frontend para reflejar el comportamiento HTTP correcto de las peticiones GET.

### 3.2 Migración y gestión de bases de datos

La persistencia fue adaptada para operar con PostgreSQL en Render. Los servicios que requieren datos dejaron de depender de una base local y utilizan una cadena de conexión obtenida desde DATABASE\_URL. La configuración de Npgsql se mantuvo fuera del código sensible y pasó al entorno de despliegue. Esta separación permite reproducibilidad entre desarrollo, CI y producción y evita almacenar credenciales en el repositorio. (PostgreSQL Global Development Group, 2026)

- Creación de la instancia muebles-postgres en Render.
- Uso de hostname, puerto, database, username y password administrados por Render.
- Configuración de DATABASE\_URL en los servicios que acceden a PostgreSQL.
- Validación de CatalogService mediante /health mostrando database: PostgreSQL.
- Prueba funcional de autenticación y catálogo contra la base desplegada.

### 3.3 Mejoras de arquitectura y conectividad

- API Gateway configurado con AUTH\_SERVICE\_URL, CATALOG\_SERVICE\_URL, INVENTORY\_SERVICE\_URL, CART\_SERVICE\_URL, ORDER\_SERVICE\_URL y PAYMENT\_SERVICE\_URL.
- Puerto del gateway gestionado mediante PORT=9090.
- CORS ampliado para permitir el origen real del frontend desplegado en Render.
- Corrección del frontend para no enviar Content-Type: application/json en GET sin cuerpo, reduciendo preflight innecesario.
- VITE\_API\_URL de producción configurada para apuntar al API Gateway de Render.

### 3.4 Contenerización y registro de imágenes

Se consolidó la construcción de ocho imágenes Docker: AuthService, CatalogService, InventoryService, CartService, OrderService, PaymentService, API Gateway y Frontend. GitHub Actions utiliza Docker Buildx y publica las imágenes en GHCR. Los nombres fueron normalizados a minúsculas para cumplir los requisitos del registry.

Cada imagen se publica con el tag latest y con el SHA del commit, proporcionando una referencia de producción y otra inmutable para trazabilidad.

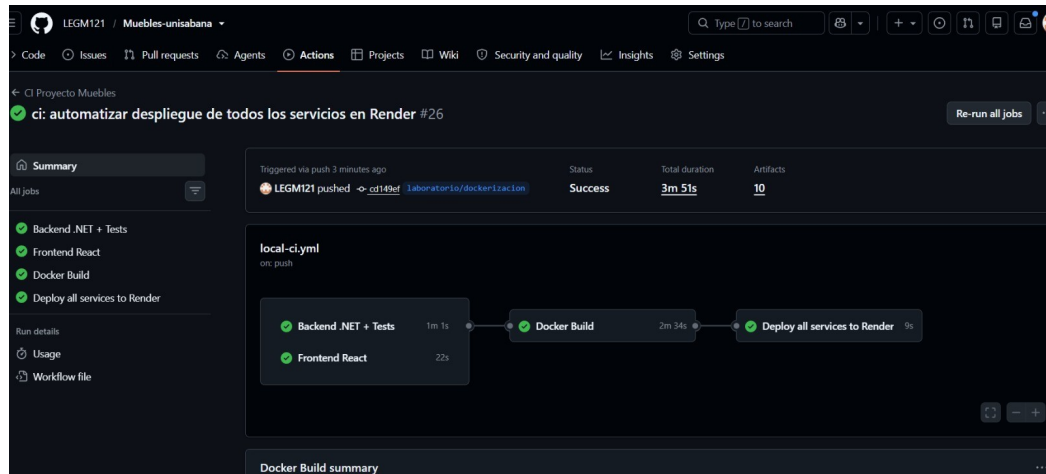
### 3.5 Automatización CI/CD

El workflow `.github/workflows/local-ci.yml` evolucionó desde un pipeline de compilación y pruebas a un flujo que también construye y publica imágenes, y finalmente dispara Deploy Hooks de Render. El despliegue se ejecuta solamente después de que Docker Build finaliza correctamente. (GitHub, 2026)

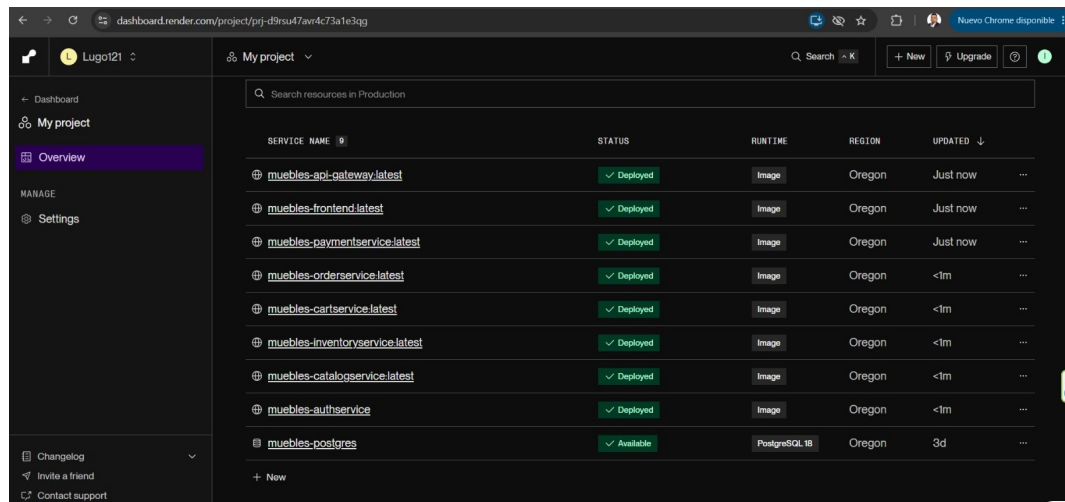
- Backend .NET + Tests automatizado.
- Frontend React: instalación, pruebas y build automatizados.
- Docker Build de ocho componentes.
- Push automático de imágenes a GHCR.
- Deploy automático de ocho servicios mediante secretos

RENDER\_\*\_DEPLOY\_HOOK.

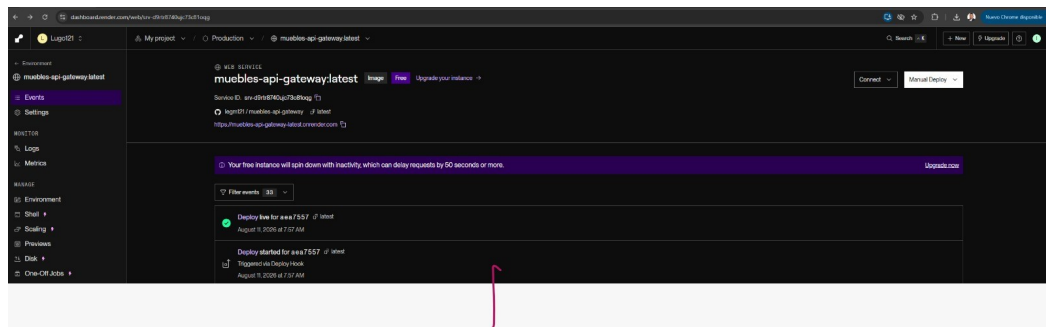
- Corrección de la estructura YAML para separar deploy-render como job independiente.



**Figura 1. GitHub Actions: pruebas, Docker Build y despliegue automático a Render.**



**Figura 2. Render: ocho servicios desplegados y PostgreSQL disponible después del pipeline.**



**Figura 3. Evidencia de Render: despliegue iniciado mediante Deploy Hook.**

### **3.6 Ajustes de estabilidad detectados durante la automatización**

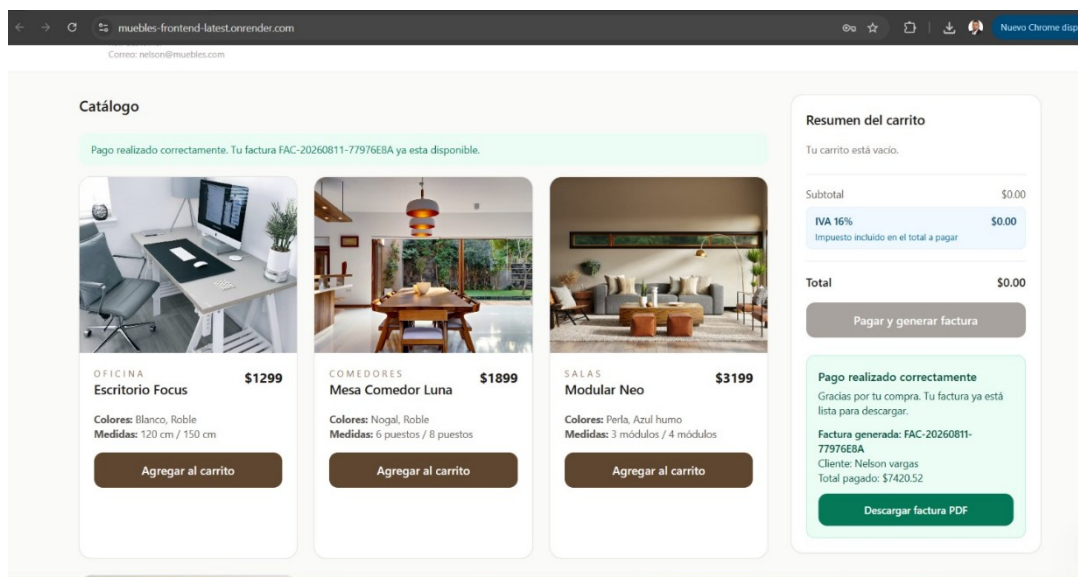
La automatización permitió detectar un problema real que no era visible con solo comprobar que el Deploy Hook había sido aceptado. CatalogService falló en Render con exit status 139 durante la inicialización de .NET. La traza apuntó al mecanismo PhysicalFileWatcher. Se añadió `DOTNET_USE_POLLING_FILE_WATCHER=1` al runtime del Dockerfile de CatalogService.

Antes de publicar el cambio se construyó y ejecutó la imagen localmente. El endpoint `/health` respondió HTTP 200 y reportó CatalogService con PostgreSQL. Posteriormente el pipeline reconstruyó la imagen, la publicó y Render volvió a mostrar CatalogService como Deployed.

### **3.7 Validación funcional en producción**

La aplicación fue validada de extremo a extremo sobre Render. La evidencia muestra productos cargados, una compra procesada correctamente y una factura generada y disponible para descarga en PDF. Esto confirma que la integración frontend → gateway → microservicios → PostgreSQL funciona en el entorno desplegado.





**Figura 4. Evidencia E2E: pago correcto y factura PDF disponible desde el frontend desplegado.**

#### 4. Estado frente a la rúbrica de la actividad

La actividad solicita: crear repositorio Git, definir branching, aplicar versionamiento semántico, buenas prácticas de commits, pipeline CI/CD (build, test, deploy), automatizar procesos clave y documentar la implementación. El entregable exige un documento técnico README.md de 5–7 páginas con repositorio, pipeline funcional, evidencias y modelo de gestión de configuración.

| Requisito                             | Estado                        | Evidencia actual                                                       | Acción pendiente                                                                   |
|---------------------------------------|-------------------------------|------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| 1. Repositorio Git                    | Cumplido                      | Repositorio GitHub y rama de trabajo.                                  | Documentar URL/estructura en README.                                               |
| 2. Estrategia de branching            | Parcial                       | Uso de laboratorio/dockerización.                                      | Formalizar el modelo: propósito de ramas, integración y reglas.                    |
| 3. Versionamiento semántico           | Pendiente                     | Existe trazabilidad latest + SHA, pero no sustituye SemVer.            | Crear release/tag v1.0.0 y política MAJOR.MINOR.PATCH.                             |
| 4. Buenas prácticas de commits        | Cumplido                      | Commits con prefijos ci:, fix:, test:.                                 | Documentar la convención adoptada.                                                 |
| 5. Pipeline CI/CD build, test, deploy | Cumplido con mejora pendiente | Build, tests, Docker, GHCR y Deploy Hooks funcionando.                 | Agregar smoke tests post-deploy para verificar producción.                         |
| 6. Automatizar procesos clave         | Avanzado                      | Construcción, pruebas, publicación y despliegue de 8 servicios.        | Automatizar health/smoke tests y, opcionalmente, rollback.                         |
| 7. Documentar implementación          | Pendiente                     | Existe evidencia acumulada.                                            | Crear README técnico final de 5–7 páginas.                                         |
| Entregable: evidencias                | Cumplido                      | Capturas de Actions, Render, health, E2E, logs y commits.              | Seleccionar y organizar evidencias finales.                                        |
| Modelo de gestión de configuración    | Parcial                       | Variables de entorno, secretos GitHub, GHCR y Render ya implementados. | Describir formalmente configuración, secretos, artefactos, versionado y promoción. |

## 5. Qué falta para cerrar la rúbrica

### 5.1 Smoke tests post-deploy - prioridad crítica

El pipeline actualmente confirma que Render acepta los Deploy Hooks, pero el incidente de CatalogService demostró que un job de deploy puede quedar verde aunque un contenedor falle después. Se recomienda agregar un job final que espere el arranque y compruebe HTTP 200 en /health de los seis microservicios, /health del gateway, /api/catalog y el frontend. (National Institute of Standards and Technology [NIST], 2024)

Resultado esperado del pipeline final:

**Tests → Docker → GHCR → Render Deploy → Smoke Tests → Producción verificada**

### 5.2 Versionamiento semántico - requerido por la actividad

Debe formalizarse SemVer. Se recomienda declarar la primera versión estable como v1.0.0 y conservar simultáneamente el tag latest y el SHA para trazabilidad. Las correcciones compatibles incrementarían PATCH; nuevas funciones compatibles, MINOR; y cambios incompatibles de API, MAJOR. (Preston-Werner, 2013)

### 5.3 Modelo de gestión de configuración - requerido en el entregable

- Código fuente: GitHub como fuente de verdad.
- Ramas: documentar la función de laboratorio/dockerización y la estrategia de integración.
- Artefactos: imágenes Docker en GHCR con latest, SHA y futuro SemVer.
- Secretos: GitHub Actions Secrets para Deploy Hooks; credenciales de base de datos en Render.

- Configuración: variables de entorno por servicio, evitando secretos hardcodedos.
- Promoción: build/test → GHCR → Render → smoke test.
- Recuperación: documentar rollback a un deployment/imagen anterior.

#### **5.4 Documento técnico README.md de 5–7 páginas**

El entregable explícito de la actividad todavía debe prepararse en formato README.md. Puede construirse a partir de este informe, condensando arquitectura, estrategia Git, SemVer, pipeline, configuración, evidencias, incidencias resueltas y conclusiones.

#### **6.1 Línea de tiempo técnica que debe quedar documentada**

- Estado inicial: servicios orientados a ejecución local, uso de localhost, puertos fijos y PostgreSQL local mediante Docker Compose.
- Ajustes de código: variables de entorno, DATABASE\_URL/Npgsql, rutas del API Gateway, CORS, VITE\_API\_URL y comportamiento de headers HTTP.
- Pruebas: validación de backend .NET y frontend; frontend con 6 archivos de prueba y 27 pruebas aprobadas.
- Contenerización: Dockerfiles por componente y construcción reproducible de ocho imágenes.
- Registro: publicación automática en GHCR con tags latest y SHA del commit.
- Migración a nube: PostgreSQL administrado en Render y ocho servicios Web desplegados desde imágenes GHCR.
- Automatización: GitHub Actions ejecuta pruebas, Docker Build, push a GHCR y Deploy Hooks para Render.

- Validación E2E: login, catálogo, carrito, orden, pago y generación de factura PDF.
- Incidencia y mejora: CatalogService presentó exit status 139; se estabilizó el runtime con DOTNET\_USE\_POLLING\_FILE\_WATCHER=1 y se verificó /health HTTP 200.

## **6.2 Evidencia de migración de base de datos y configuración**

Para demostrar la migración no basta con afirmar que PostgreSQL está en Render. Conviene mostrar el antes y el después. En el estado local, Docker Compose contenía DATABASE\_URL con host interno del contenedor PostgreSQL. En producción, la configuración se trasladó a variables de entorno de Render y a una instancia administrada muebles-postgres. No deben publicarse contraseñas, URLs privadas completas ni secretos en el README.

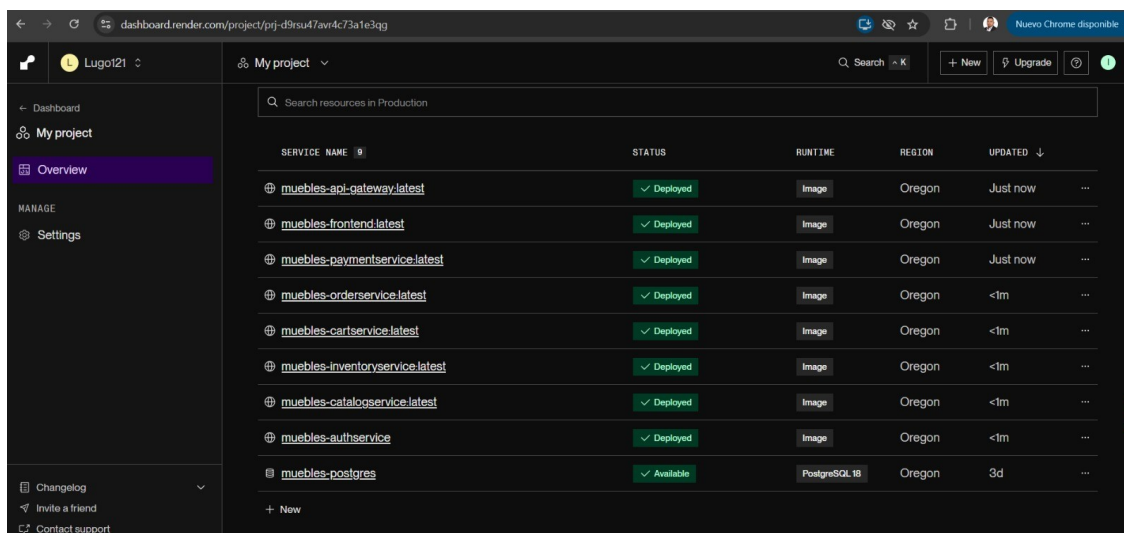
```

$ ls
README.md backend/ database/ defectos_integracion.md docker-compose.yml docs/ frontend/ node-api-gateway/ reports/ root.gitignore scripts/ shared/

$ docker compose config
name: proyecto-muebles
services:
  authservice:
    command:
      - sh
      - -c
      - dotnet run --project AuthService.Api/AuthService.Api.csproj --urls=http://0.0.0.0:8081
    container_name: authservice
    depends_on:
      postgres:
        condition: service_healthy
        required: true
    environment:
      ASPNETCORE_ENVIRONMENT: Development
      DATABASE_URL: Host=proyecto-muebles-postgres;Port=5432;Database=muebles_db;Username=postgres;Password=postgres
    image: mcr.microsoft.com/dotnet/sdk:8.0
    networks:
      app-network: null
    ports:
      - mode: ingress
        target: 8081
        published: "8081"
        protocol: tcp
    volumes:
      - type: bind
        source: C:\Proyecto\Proyecto-Muebles4-main\backend\services\AuthService
        target: /src
        bind: {}
    working_dir: /src
  cartservice:
    command:
      - sh
      - -c
      - dotnet run --project CartService.Api/CartService.Api.csproj --urls=http://0.0.0.0:8083
    container_name: cartservice
    depends_on:
      postgres:
        condition: service_healthy
        required: true
    environment:
      ASPNETCORE_ENVIRONMENT: Development
      DATABASE_URL: Host=proyecto-muebles-postgres;Port=5432;Database=muebles_db;Username=postgres;Password=postgres
    image: mcr.microsoft.com/dotnet/sdk:8.0
    networks:
      app-network: null
    ports:
      - mode: ingress
        target: 8083
        published: "8083"
        protocol: tcp
    volumes:
      - type: bind
        source: C:\Proyecto\Proyecto-Muebles4-main\backend\services\CartService
        target: /src
        bind: {}
    working_dir: /src
  catalogservice:
    command:
      - sh

```

**Figura 5. Evidencia del estado local: Docker Compose, PostgreSQL y DATABASE\_URL entre contenedores.**



The screenshot shows the Render dashboard for a project named 'My project'. The left sidebar contains navigation links: 'Dashboard', 'My project', 'Overview' (selected), 'MANAGE', 'Settings', 'Changelog', 'Invite a friend', and 'Contact support'. The main area displays a table of services in the 'Production' environment.

| SERVICE NAME                                    | STATUS      | RUNTIME       | REGION | UPDATED  |     |
|-------------------------------------------------|-------------|---------------|--------|----------|-----|
| <a href="#">muebles-api-gateway:latest</a>      | ✓ Deployed  | Image         | Oregon | Just now | ... |
| <a href="#">muebles-frontend:latest</a>         | ✓ Deployed  | Image         | Oregon | Just now | ... |
| <a href="#">muebles-paymentservice:latest</a>   | ✓ Deployed  | Image         | Oregon | Just now | ... |
| <a href="#">muebles-orderservice:latest</a>     | ✓ Deployed  | Image         | Oregon | <1m      | ... |
| <a href="#">muebles-cartservice:latest</a>      | ✓ Deployed  | Image         | Oregon | <1m      | ... |
| <a href="#">muebles-inventoryservice:latest</a> | ✓ Deployed  | Image         | Oregon | <1m      | ... |
| <a href="#">muebles-catalogservice:latest</a>   | ✓ Deployed  | Image         | Oregon | <1m      | ... |
| <a href="#">muebles-authservice</a>             | ✓ Deployed  | Image         | Oregon | <1m      | ... |
| <a href="#">muebles-postgres</a>                | ✓ Available | PostgreSQL 18 | Oregon | 3d       | ... |

At the bottom of the table, there is a '+ New' button.

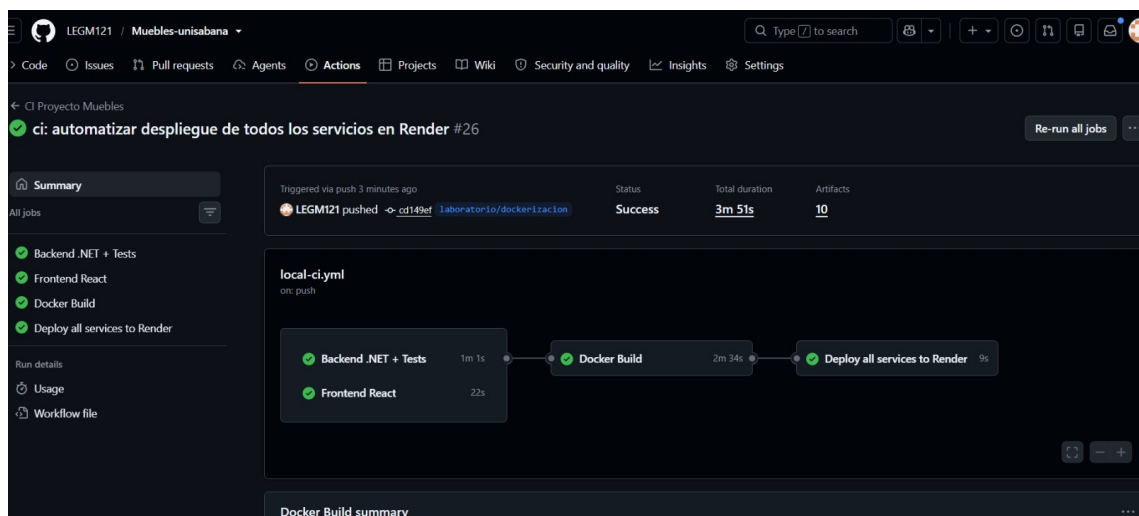
**Figura 6. Estado en nube: ocho servicios Deployed y PostgreSQL disponible en Render.**

En la documentación final, acompañar estas capturas con una explicación como:

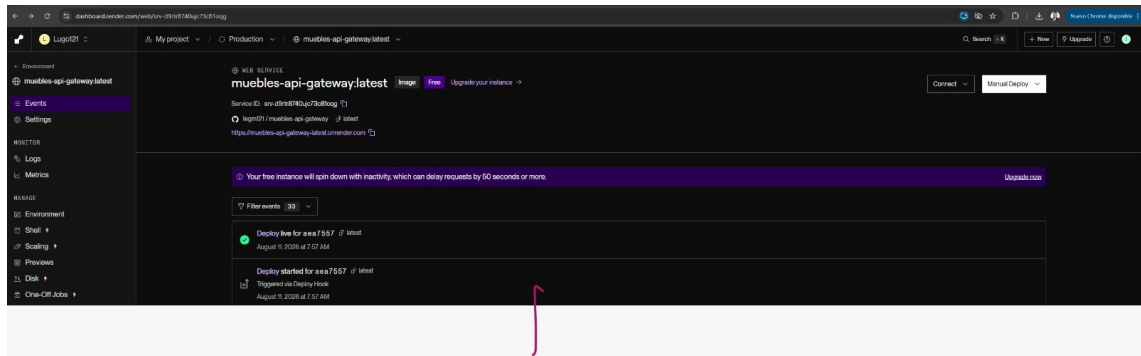
*“La persistencia pasó de una dependencia local de Docker Compose a PostgreSQL administrado en Render. Las credenciales y cadenas de conexión se desacoplaron del código mediante variables de entorno, manteniendo DATABASE\_URL como contrato de configuración para los servicios .NET/Npgsql.”*

### 6.3 Evidencia del pipeline y automatización

El README debe explicar la dependencia entre jobs: backend y frontend se validan primero; Docker Build se ejecuta después; finalmente el job de despliegue invoca los Deploy Hooks. Esto demuestra que el despliegue no depende de una acción manual del operador.



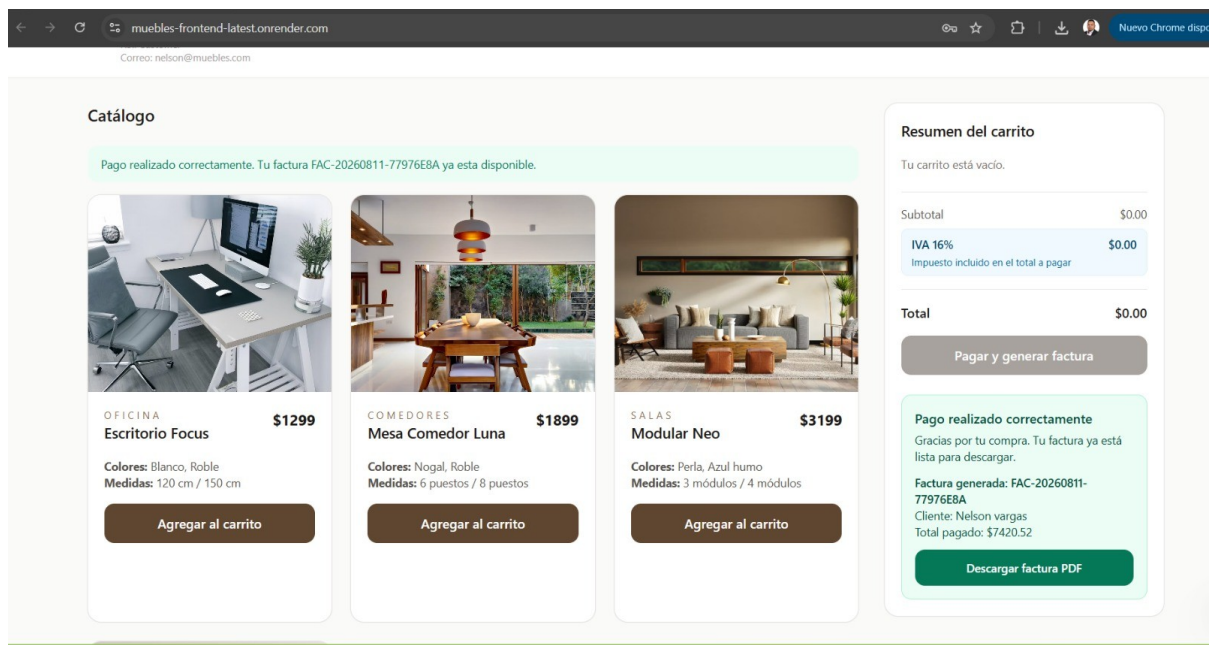
**Figura 7. GitHub Actions: Backend + Tests, Frontend, Docker Build y Deploy all services to Render.**



**Figura 8. Render confirma que el despliegue fue “Triggered via Deploy Hook”.**

## 6.4 Evidencia funcional y de negocio

La evidencia E2E debe cerrar la historia técnica: no solo se desplegaron contenedores, sino que la solución completa procesa el flujo de negocio. La captura de pago y factura es especialmente útil porque implica comunicación entre frontend, gateway, servicios de dominio y persistencia.

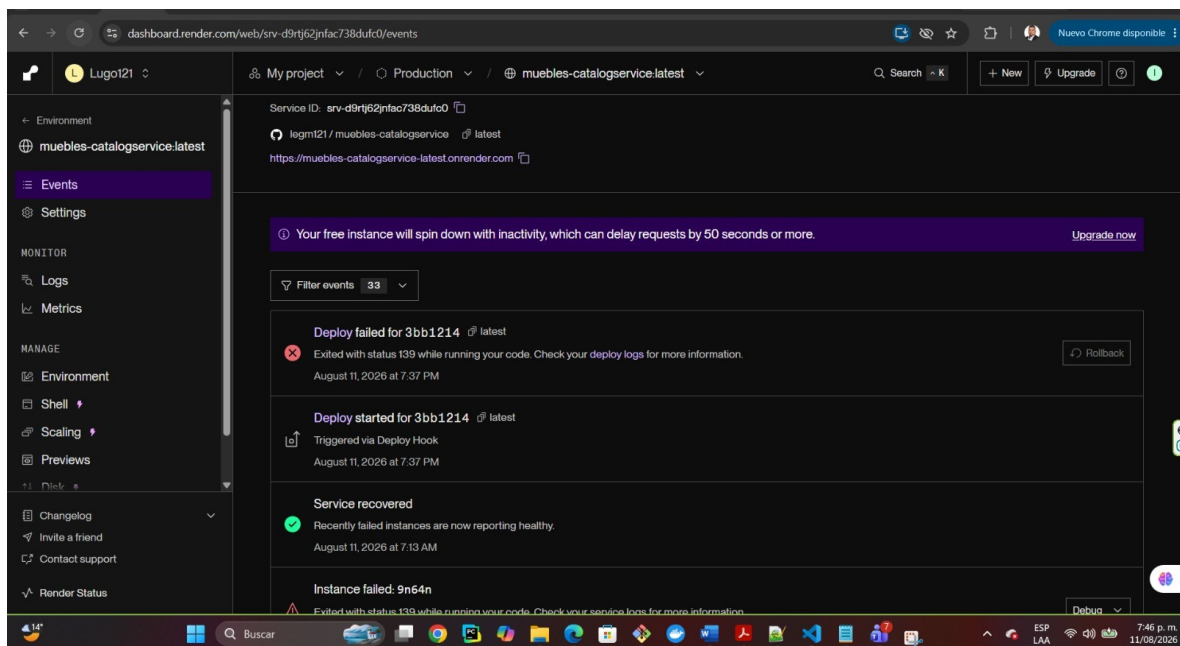


**Figura 9. Evidencia E2E: catálogo operativo, pago exitoso y factura PDF generada.**

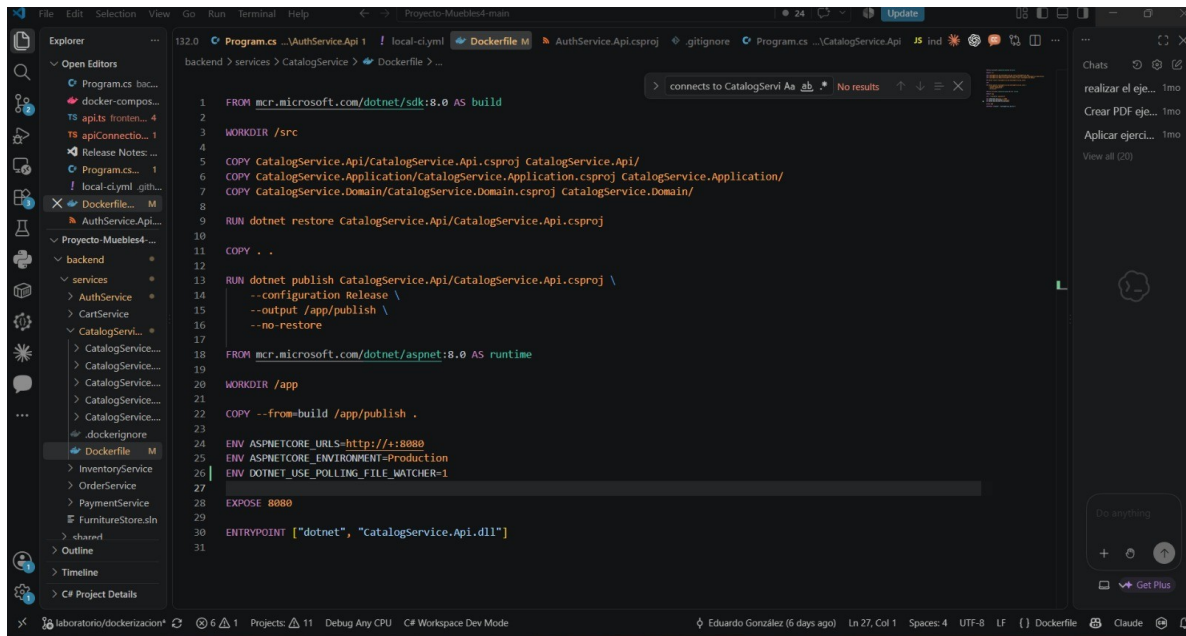


## 6.5 Evidencia de resolución de incidentes y mejora del código

También es valioso documentar un incidente real y su resolución. CatalogService llegó a fallar en Render con status 139. La investigación revisó los logs de arranque .NET, el Dockerfile y el comportamiento de FileWatcher. Se agregó `DOTNET_USE_POLLING_FILE_WATCHER=1` al runtime, se construyó la imagen local y se validó el endpoint `/health` antes de publicar.



**Figura 10. Incidente real: CatalogService falla en Render con exit status 139.**



**Figura 11. Ajuste del Dockerfile de CatalogService para estabilizar FileWatcher en contenedor.**

```

MINGW64/G/Proyecto/Proyecto-Muebles4-main
LENOVO8DESKTOP-9QP3NGF MINGW64 /c/Proyecto/Proyecto-Muebles4-main (laboratorio/dockerizacion)
$ docker build -t muebles-catalogservice:test backend/services/CatalogService
[+] Building 10.0s (17/17) FINISHED
=> [internal] load build definition from Dockerfile
=> transferring dockerfile: 835B
=> [internal] load metadata for mcr.microsoft.com/dotnet/aspnet:8.0
=> [internal] load metadata for mcr.microsoft.com/dotnet/sdk:8.0
=> [internal] load .dockerignore
=> transferring context: 138B
=> [runtime 1/3] FROM mcr.microsoft.com/dotnet/aspnet:8.0@sha256:b0beb9cc1dee1cb0749796110d4734292071b814207ad0d4f40611f7db04f7b
=> resolve mcr.microsoft.com/dotnet/aspnet:8.0@sha256:b0beb9cc1dee1cb0749796110d4734292071b814207ad0d4f40611f7db04f7b
=> sha256:8f6ea256f4f50387ea415ec03923e73984139503462ad7c4b1ab7d32276d0c13 3.33kB / 3.33kB
=> sha256:16065d0ce56265890a97dd21d625dcbcf5d1a2a1cd2c9c188527a6d9b9f9dce 11.10MB / 11.10MB
=> sha256:94720aab89da32a22539bd24d62f1d88bb64cbbd49f359425acafda0a7160077 18.74MB / 18.74MB
=> sha256:7216a21996f1f66318b54231e4c2e39c8986227535647628d466e26c4748387a 165B / 165B
=> sha256:426e0e96f5a174291a02648d1eaadd3fe5f28c62d0b0e702e74100de4da13b3 32.26MB / 32.26MB
=> extracting sha256:94720aab89da32a22539bd24d62f1d88bb64cbbd49f359425acafda0a7160077
=> extracting sha256:8f6ea256f4f50387ea415ec03923e73984139503462ad7c4b1ab7d32276d0c13
=> extracting sha256:426e0e96f5a174291a02648d1eaadd3fe5f28c62d0b0e702e74100de4da13b3
=> extracting sha256:7216a21996f1f66318b54231e4c2e39c8986227535647628d466e26c4748387a
=> extracting sha256:16065d0ce56265890a97dd21d625dcbcf5d1a2a1cd2c9c188527a6d9b9f9dce
=> [build 1/8] FROM mcr.microsoft.com/dotnet/sdk:8.0@sha256:fc69dc5e0c9789adaac5c8efce71ead4d016a51318667c4f26ce93574b1b9403
=> resolve mcr.microsoft.com/dotnet/sdk:8.0@sha256:fc69dc5e0c9789adaac5c8efce71ead4d016a51318667c4f26ce93574b1b9403
=> [internal] load build context
=> transferring context: 2.44kB
=> CACHED [build 2/8] WORKDIR /src
=> CACHED [build 3/8] COPY CatalogService.Api/CatalogService.Api.csproj CatalogService.Api/
=> CACHED [build 4/8] COPY CatalogService.Application/CatalogService.Application.csproj CatalogService.Application/
=> CACHED [build 5/8] COPY CatalogService.Domain/CatalogService.Domain.csproj CatalogService.Domain/
=> CACHED [build 6/8] RUN dotnet restore CatalogService.Api/CatalogService.Api.csproj
=> [build 7/8] COPY .
=> [build 8/8] RUN dotnet publish CatalogService.Api/CatalogService.Api.csproj --configuration Release --output /app/publish --no-restore
=> [runtime 2/3] WORKDIR /app
=> [runtime 3/3] COPY --from=build /app/publish .
=> exporting to image
=> exporting layers
=> exporting manifest sha256:282be91fa4e794e15635a986695615dcf401df71366f17ea5c076adbd29361f8
=> exporting config sha256:28d4de9e1b0019c4da7292e1cdc067056e1d365356bf4e83df155ad351a858f
=> exporting attestation manifest sha256:7b37672e1debcbefbb6e25e54d9e483d982641bed07f6f02baa8545d727ca61
=> exporting manifest list sha256:22bac1e103cf764445454a81b7cf777d32fde46edcbcf4575ee3692b93845b
=> naming to docker.io/library/muebles-catalogservice:test
=> unpacking to docker.io/library/muebles-catalogservice:test

LENOVO8DESKTOP-9QP3NGF MINGW64 /c/Proyecto/Proyecto-Muebles4-main (laboratorio/dockerizacion)
$ docker run --rm -d \
  --name catalog-test \
  -p 8082:8080 \
  muebles-catalogservice:test
7d7053aef8e146cc639901abb2e045d4d7c786241f380daec459f5fe8e3d2ba1

LENOVO8DESKTOP-9QP3NGF MINGW64 /c/Proyecto/Proyecto-Muebles4-main (laboratorio/dockerizacion)
$ curl -I http://localhost:8082/health
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Date: Wed, 12 Aug 2026 00:59:04 GMT
Server: Kestrel
Transfer-Encoding: chunked

{"service":"CatalogService","database":"PostgreSQL"}
LENOVO8DESKTOP-9QP3NGF MINGW64 /c/Proyecto/Proyecto-Muebles4-main (laboratorio/dockerizacion)
$

```

**Figura 12. Validación local posterior: imagen construida y /health de CatalogService responde HTTP 200.**

## 8.6 Matriz de evidencias recomendada para el README

| Tema                 | Evidencia                                      | Qué demuestra                               | Criticidad |
|----------------------|------------------------------------------------|---------------------------------------------|------------|
| Git y branching      | Captura de rama y commits                      | Trazabilidad y estrategia de cambios        | Alta       |
| Pruebas              | GitHub Actions / salida npm test y dotnet test | Quality gate previo a Docker                | Crítica    |
| Migración PostgreSQL | Compose local + Render PostgreSQL + /health    | Cambio de persistencia local a administrada | Crítica    |
| GHCR                 | Packages con 8 imágenes                        | Artefactos versionados y reutilizables      | Alta       |
| Render               | Overview con servicios Deployed                | Arquitectura ejecutándose en nube           | Crítica    |
| Deploy Hook          | Evento Triggered via Deploy Hook               | CD sin Manual Deploy                        | Crítica    |

|                   |                                         |                                           |         |
|-------------------|-----------------------------------------|-------------------------------------------|---------|
| E2E               | Pago y factura PDF                      | Funcionamiento integrado de negocio       | Crítica |
| Incidente Catalog | Failed deploy + corrección + health 200 | Capacidad de diagnóstico y endurecimiento | Alta    |
| SemVer            | Release v1.0.0 (pendiente)              | Cumplimiento explícito de la rúbrica      | Crítica |
| Smoke tests       | Job post-deploy (pendiente)             | Verificación automática de producción     | Crítica |

## 6.7 Información vital que debe quedar escrita, no solo en capturas

- Nombre y propósito de cada microservicio y su relación con el API Gateway.
- Mapa de puertos locales y cómo en Render se reemplazan por URLs/variables de entorno.
- Variables de entorno utilizadas, indicando el nombre pero nunca el valor de secretos.
- Qué servicios usan PostgreSQL y cómo se obtiene la cadena DATABASE\_URL.
- Estrategia de tags de imágenes: latest + SHA; agregar SemVer v1.0.0 para cerrar la rúbrica.
- Orden de jobs de GitHub Actions y condición de dependencia mediante needs.
- Lista de secretos RENDER\_\*\_DEPLOY\_HOOK, documentando solo sus nombres.
- Limitación de instancias Free de Render: cold start/spin down y posible demora inicial.
- Problemas encontrados (CORS, rutas, headers GET, Catalog status 139) y cómo fueron resueltos.
- Prueba E2E final y resultado esperado.

- Pendientes: smoke tests post-deploy, SemVer formal, README final y opcionalmente Trivy/Sonar/Grafana.

## 6.8 Estructura sugerida de la carpeta de documentación

```
docs/
    evidencias/
        01-git-branching/
        02-tests/
        03-postgresql-migracion/
        04-ghcr/
        05-render/
        06-cicd/
        07-e2e/
        08-incidentes/
    arquitectura.md
    modelo-configuracion.md
README.md
```

Esta organización facilita relacionar cada requisito de la rúbrica con una evidencia concreta y evita que el README se convierta en una colección desordenada de capturas.

## **Definir estrategia de branching**

### **GitHub Flow Adaptado**

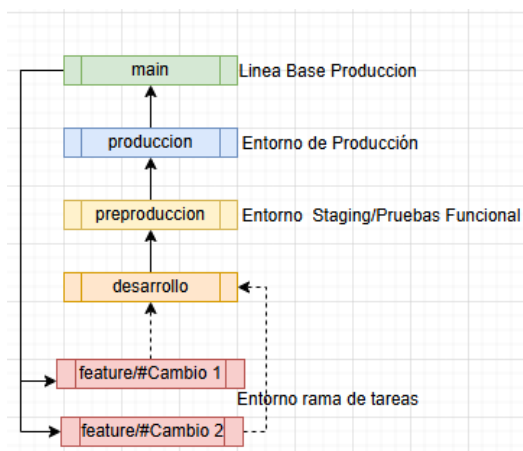
Al usar un monorepo (React + Node.js + .NET 8) y desplegar automáticamente a Render a través de GitHub Actions, necesitamos un flujo que proteja la producción pero que no ralentice el mantenimiento mensual

### **Estructura de Ramas Permanentes**

- **main (o master):** Representa el entorno como línea base de Producción (última versión que se encuentra funcional en producción).
- **produccion:** Representa el entorno de Producción (lo que ven los clientes finales al comprar muebles). Está conectada al despliegue automático de Render para producción.
- **preproduccion:** Representa el entorno de Staging/Pruebas para el equipo funcional de la empresa, aquí se integran los cambios de la rama de desarrollo
- **Desarrollo:** Representa el entorno de Staging/Pruebas para el ambiente de desarrolladores. Aquí se integran los cambios de los desarrolladores para probar que el frontend, el gateway y los servicios de .NET funcionen correctamente juntos antes del despliegue mensual.

### **Flujo de Trabajo**

Para el mantenimiento mensual o la adición de funciones o controles de cambio, se sigue el siguiente flujo:



### Creación de la rama de tarea

Cada desarrollador crea una rama temporal desde main para trabajar en su servicio asignado con fin de gestionar ajustes, mantenimiento o controles de cambio:

`feature/ajuste-carrifox/auth-gateway`

Al modificar el código, se debe utilizar prefijos en los mensajes de commit para saber qué parte del monorepo se afectó, por ejemplo: `feat(frontend): ui del carrito` o `fix(backend): endpoint de facturación`.

### Integración y Pruebas Locales (Docker)

Antes de subir el código, el desarrollador levanta el entorno completo usando su automatización de Docker (`docker-compose`) localmente. Esto asegura que los cambios en el backend de .NET 8 no rompan el Gateway de Node o la app de React.

### Pull Request a Rama de cambios

El desarrollador sube su rama local a GitHub y abre un Pull Request (PR) hacia la rama de cambio y así poderla llevar a las siguientes ramas. Otro desarrollador o aprobador revisa el código (Code Review), lo aprueba y se fusiona (Merge).

Despliegue en Entorno de Desarrollo

Al fusionarse en desarrollo, GitHub Actions compila los contenedores Docker y los despliega automáticamente en un servicio de pruebas de desarrollo en Render. Aquí los desarrolladores entran a la web y validan el flujo completo: iniciar sesión -> agregar mueble al carrito -> generar factura.

### **Despliegue en Entorno de Pruebas (Render)**

Al fusionarse en preproduccion, GitHub Actions compila los contenedores Docker y los despliega automáticamente en un servicio de pruebas en Render. Aquí ambos desarrolladores entran a la web y validan el flujo completo: iniciar sesión -> agregar mueble al carrito -> generar factura.

### **El Despliegue Paso a produccion**

Cuando todas las tareas planificadas están integradas y probadas en preproduccion, se abre un único PR de development hacia produccion. Al aprobarlo, Render actualizará la aplicación en producción de manera limpia y sin errores imprevistos.

### **Generar linea base de produccion**

Cunado se valida que el despliegue en producción su funcionalidad cumple con los requerimientos solicitados , se genera una linea base a main a partir de produccion

### **Prácticas De Commits**

Para el proyecto (**Monorepo**) y un equipo de **2 desarrolladores**, la mejor decisión es implementar **Conventional Commits** adaptado a monorrepos. Esto permitirá entender de un vistazo qué servicio se modificó (React, Node o .NET 8) y automatizar el historial de cambios.

Cada commit debe seguir esta estructura:

text<tipo>(<servicio-afectado>): <descripción corta en imperativo>



## Commits

main

All users
 All time

Commits on Aug 12, 2026

Merge pull request #1 from LEGM121/produccion
 Verified
 c77232

Merge branch 'main' into produccion
 Verified
 30392f6

Commits on Aug 11, 2026

fix: estabilizar CatalogService en contenedor
 f08da0d

ci: automatizar despliegue de todos los servicios en Render
 cd149ef

ci: automatizar despliegue del gateway en Render 2
 496b091

ci: automatizar despliegue del gateway en Render
 a27a43f

## Versionamiento Semántico

Se implementa el versionamiento semántico centralizado en el código, con versión base 1.0.0 y sincronización automática para frontend, gateway y servicios .NET.

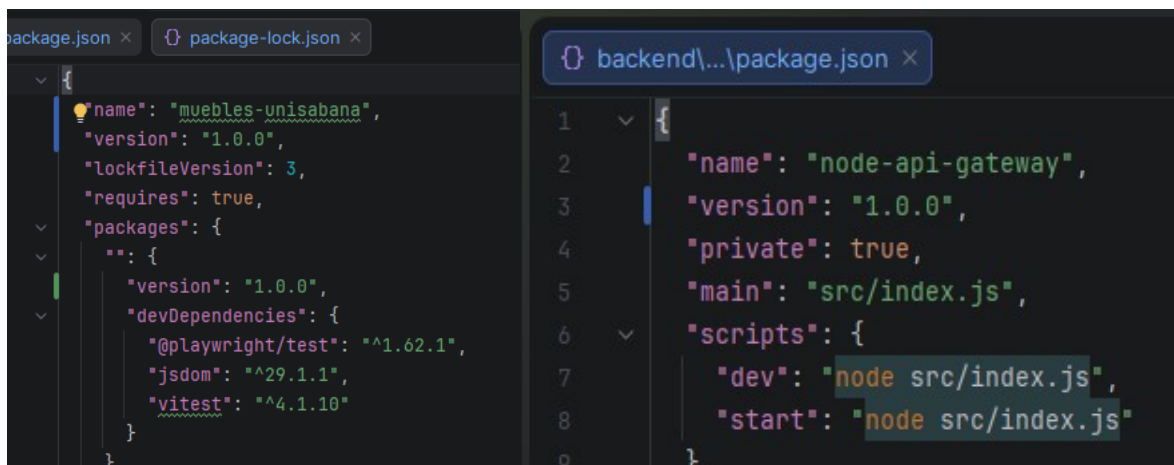
**1. package.json (raíz):** agregué version, version:sync, version:patch, version:minor, version:major.

```

package.json
1  {
2    "name": "muebles-unisabana",
3    "private": true,
4    "version": "1.0.0",
5    "scripts": {
6      "version:sync": "node scripts/sync-version.js",
7      "version:patch": "npm version patch --no-git-tag-version && npm run version:sync",
8      "version:minor": "npm version minor --no-git-tag-version && npm run version:sync",
9      "version:major": "npm version major --no-git-tag-version && npm run version:sync",
10   },
11   "devDependencies": {
12     "@playwright/test": "^1.62.1",
13     "jsdom": "^29.1.1",
14     "vitest": "^4.1.10"
15   }
16 }
  
```

## 2. scripts\sync-version.js: nuevo script que toma la versión raíz y la sincroniza en:

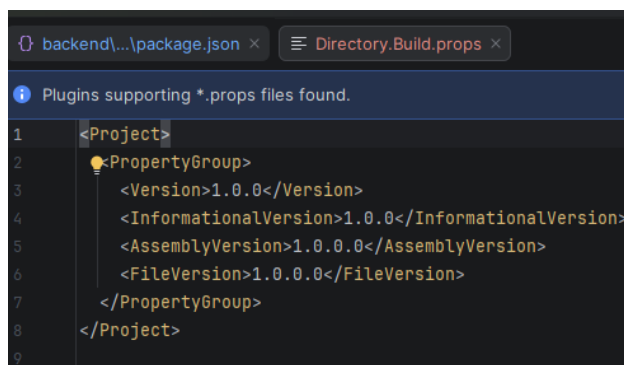
- frontend\package.json y frontend\package-lock.json
- backend\node-api-gateway\package.json y backend\node-api-gateway\package-lock.json
- backend\services\Directory.Build.props
- tags de docker-compose.lab.yml con \${APP\_VERSION:-<version>}



```
package.json x package-lock.json x
{
  "name": "muebles-unisabana",
  "version": "1.0.0",
  "lockfileVersion": 3,
  "requires": true,
  "packages": {
    "": {
      "version": "1.0.0",
      "devDependencies": {
        "@playwright/test": "^1.62.1",
        "jsdom": "^29.1.1",
        "vitest": "^4.1.10"
      }
    }
  }
}

backend\...\package.json x
1 {
2   "name": "node-api-gateway",
3   "version": "1.0.0",
4   "private": true,
5   "main": "src/index.js",
6   "scripts": {
7     "dev": "node src/index.js",
8     "start": "node src/index.js"
9 }
```

3. backend\services\Directory.Build.props: nuevo archivo para versionado .NET (Version, InformationalVersion, AssemblyVersion, AssemblyFileVersion).



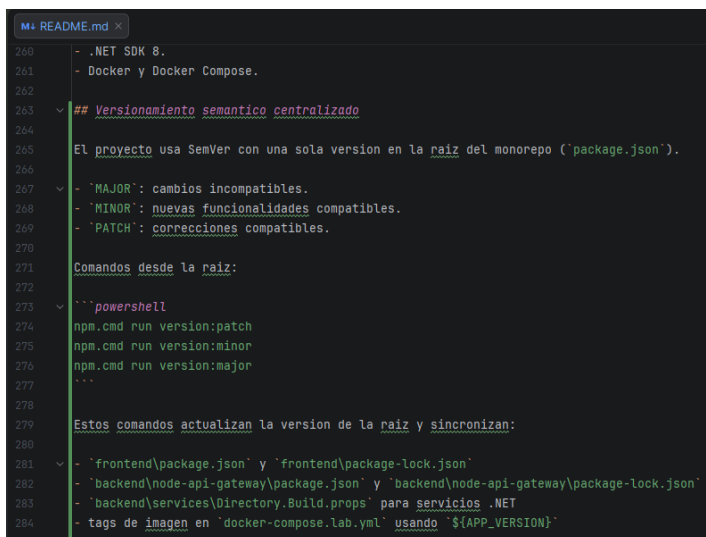
```

1 <Project>
2   <PropertyGroup>
3     <Version>1.0.0</Version>
4     <InformationalVersion>1.0.0</InformationalVersion>
5     <AssemblyVersion>1.0.0.0</AssemblyVersion>
6     <FileVersion>1.0.0.0</FileVersion>
7   </PropertyGroup>
8 </Project>

```

4. docker-compose.lab.yml y .env.lab.example: ahora usan APP\_VERSION (1.0.0 por defecto).

5. README.md: documenté el flujo de versionamiento centralizado y comandos de uso.



```

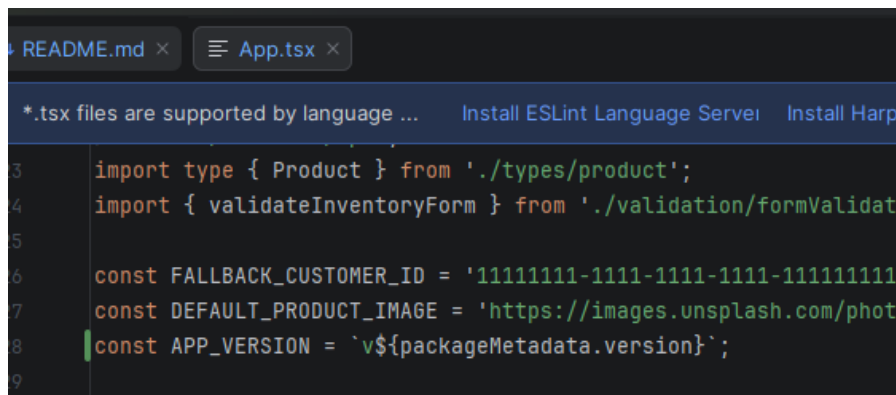
260 - .NET SDK 8.
261 - Docker y Docker Compose.
262
263 ## Versionamiento semantico centralizado
264
265 El proyecto usa SemVer con una sola version en la raiz del monorepo ('package.json').
266
267 - 'MAJOR': cambios incompatibles.
268 - 'MINOR': nuevas funcionalidades compatibles.
269 - 'PATCH': correcciones compatibles.
270
271 Comandos desde la raiz:
272
273 ```powershell
274 npm.cmd run version:patch
275 npm.cmd run version:minor
276 npm.cmd run version:major
277 ```
278
279 Estos comandos actualizan la version de la raiz y sincronizan:
280
281 - 'frontend\package.json' y 'frontend\package-lock.json'
282 - 'backend\node-api-gateway\package.json' y 'backend\node-api-gateway\package-lock.json'
283 - 'backend\services\Directory.Build.props' para servicios .NET
284 - tags de imagen en 'docker-compose.lab.yml' usando '${APP_VERSION}'

```

Se renderiza la versión frontend\src\App.tsx, mostrando: Versión desplegada: v1.0.0

Toma la versión directamente de frontend\package.json (import packageMetadata from './package.json'), así que al actualizar el versionado semántico se reflejará automáticamente en la

UI.



```

3   import type { Product } from './types/product';
4   import { validateInventoryForm } from './validation/formValidat
5
6   const FALLBACK_CUSTOMER_ID = '11111111-1111-1111-1111-1111111111
7   const DEFAULT_PRODUCT_IMAGE = 'https://images.unsplash.com/phot
8   const APP_VERSION = `v${packageMetadata.version}`;
9

```

### Actualizar Versionamiento En Aplicación

Solo necesitas ejecutar el comando de versión en la raíz del repo; eso actualiza el número y lo sincroniza para que se vea en frontend.

1. `npm.cmd run version:patch` # 1.0.0 -> 1.0.1
2. `npm.cmd run version:minor` # 1.0.0 -> 1.1.0
3. `npm.cmd run version:major` # 1.0.0 -> 2.0.0

Eso corre `version:sync` automáticamente y actualiza `package.json/package-lock` (raíz, frontend, gateway), `Directory.Build.props` y `docker-compose.lab.yml`.

### Manejo de Versionamiento:

- `version:patch`: sube el tercer número (1.0.0 → 1.0.1).

Para correcciones pequeñas/bugs sin romper compatibilidad.

- `version:minor`: sube el segundo número (1.0.0 → 1.1.0).

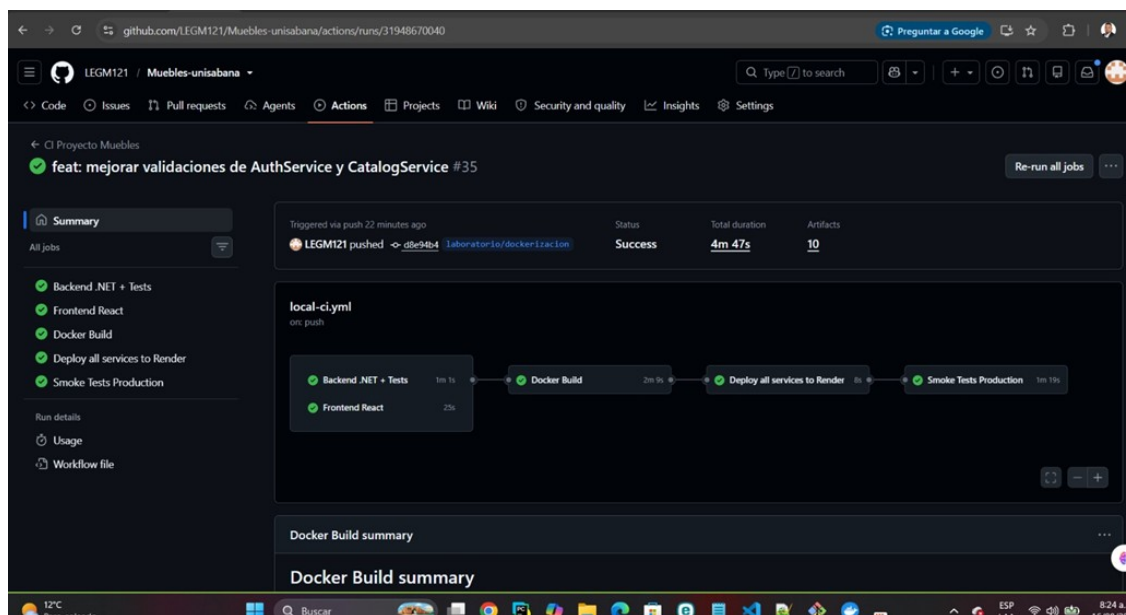
Para nuevas funcionalidades compatibles.

1. `version:major`: sube el primer número (1.0.0 → 2.0.0).

Para cambios que rompen compatibilidad (API/contratos/UI esperada).

### Otras mejoras implementadas:

| Mejora         | Antes                                                                           | Después                                                                        | Beneficio principal                                                                      |
|----------------|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| CD             | El deploy podía ser disparado sin confirmar la salud final de producción.       | Se agregaron smoke tests posteriores al despliegue.                            | Detectar fallos reales después de Render y evitar falsos positivos.                      |
| AuthService    | Se verificaban campos vacíos, pero no el formato del correo de forma homogénea. | Validación centralizada de email aplicada a register, login y forgot-password. | Evitar solicitudes inválidas antes de ejecutar lógica de autenticación o acceso a datos. |
| CatalogService | Existían reglas de obligatoriedad y precio.                                     | Se agregaron límites de 100 caracteres para nombre y 50 para categoría.        | Proteger la calidad y consistencia de los datos del catálogo.                            |



La justificación principal es eliminar el falso positivo de “despliegue exitoso”. Antes, el pipeline podía confirmar que Render había recibido la orden de despliegue aunque un servicio

fallara posteriormente durante el arranque. Con los smoke tests, el pipeline solo finaliza correctamente cuando la aplicación desplegada responde en producción.

| Control            | Riesgo mitigado                         | Resultado                                        |
|--------------------|-----------------------------------------|--------------------------------------------------|
| Dependencias needs | Desplegar código que no superó CI/build | El deploy depende del Docker Build.              |
| Secrets de Render  | Exposición de Deploy Hooks              | Hooks gestionados como secretos del repositorio. |
| Smoke tests        | Deploy verde pero aplicación caída      | El workflow falla si producción no responde.     |
| Reintentos         | Falsos fallos por cold start            | Mayor estabilidad en Render Free.                |

AuthService: validación consistente del correo electrónico

- Problema identificado

AuthService ya verificaba campos obligatorios mediante `IsNullOrWhiteSpace`, pero la validación del formato de correo no estaba centralizada ni aplicada de manera consistente en los principales flujos de autenticación. Esto permitía que valores con formato evidentemente inválido avanzaran más de lo necesario dentro del servicio.

- 2.Cambio implementado

Se incorporó la función reutilizable `IsValidEmail()`, basada en `System.Net.Mail.MailAddress`, y se aplicó después de las validaciones de obligatoriedad en los endpoints de registro, inicio de sesión y recuperación de contraseña. El orden mantiene el comportamiento existente para campos vacíos y añade una segunda barrera para el formato del correo.

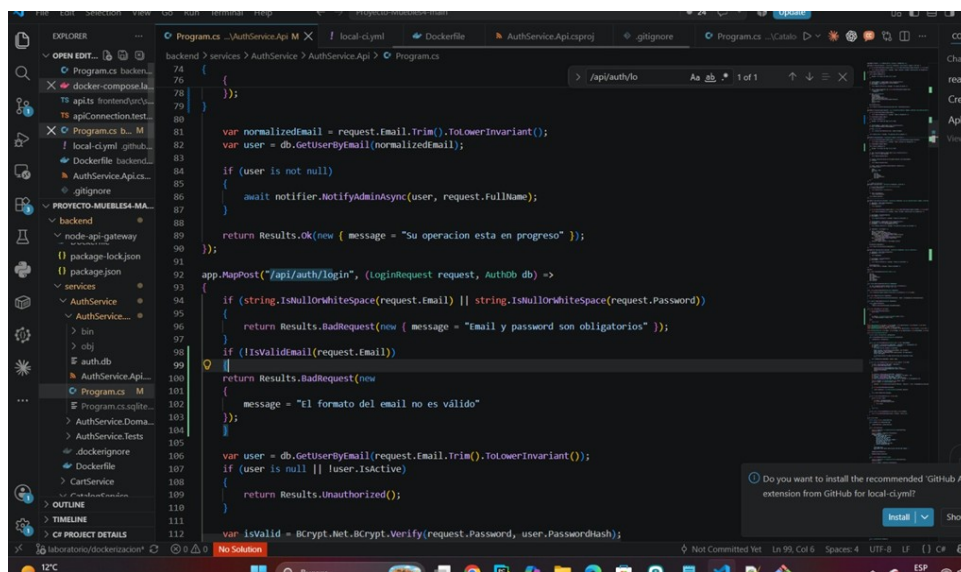
POST `/api/auth/register`: valida obligatoriedad y formato del email antes de normalizarlo y consultar usuarios.

POST /api/auth/login: valida email/password obligatorios y luego el formato del email antes de consultar AuthDb.

POST /api/auth/forgot-password: valida email obligatorio y formato antes de continuar con la recuperación.

La lógica existente de autenticación, BCrypt, roles y acceso a datos no fue reemplazada.

AuthService: validación de formato de email incorporada en el endpoint de login.



```

74 {
75 }
76 }
77 }
78 }
79 }
80 }
81 var normalizedEmail = request.Email.Trim().ToLowerInvariant();
82 var user = db.GetUserByEmail(normalizedEmail);
83
84 if (user is not null)
85 {
86     await notifier.NotifyAdminAsync(user, request.FullName);
87 }
88
89 return Results.Ok(new { message = "Su operacion esta en progreso" });
90 }
91
92 app.MapPost("/api/auth/login", (LoginRequest request, AuthDb db) =>
93 {
94     if (string.IsNullOrWhiteSpace(request.Email) || string.IsNullOrWhiteSpace(request.Password))
95     {
96         return Results.BadRequest(new { message = "Email y password son obligatorios" });
97     }
98     if (!IsValidEmail(request.Email))
99     {
100         return Results.BadRequest(new
101         {
102             message = "El formato del email no es válido"
103         });
104     }
105
106     var user = db.GetUserByEmail(request.Email.Trim().ToLowerInvariant());
107     if (user is null || !user.IsActive)
108     {
109         return Results.Unauthorized();
110     }
111
112     var isValid = BCrypt.Net.BCrypt.Verify(request.Password, user.PasswordHash);

```

## Beneficio y justificación

La mejora se justifica porque la validación debe ocurrir lo más cerca posible de la entrada del sistema. Rechazar un email inválido antes de consultar la base de datos reduce procesamiento innecesario, mejora la coherencia de las respuestas HTTP 400 y evita duplicar reglas distintas entre endpoints. Además, centralizar la comprobación facilita su mantenimiento futuro.

Verificación local del AuthService: las pruebas existentes finalizan 9/9 correctamente después de la modificación.

```

MINGW64/C:/Proyecto/Proyecto-Muebles-main
ENVIRONMENT: RQ3N3P RING64 /C:/Proyecto/Proyecto-Muebles4-main (Laboratorio/docker/izacion)
$ docker ps
Failed to connect to the docker API at npipe://npipe/docker_engine; check if the path is correct and if the daemon is running; open npipe://npipe/docker_engine: The system cannot find the file specified.
ENVIRONMENT: RQ3N3P RING64 /C:/Proyecto/Proyecto-Muebles4-main (Laboratorio/docker/izacion)
$ docker ps
CONTAINER ID        IMAGE                                     COMMAND                  CREATED              STATUS              PORTS              NAMES
7d443b58f13        proyecto-muebles/api-gateway:lab        "/docker-entrypoint.s..." 9 days ago          Up 27 seconds      0.0.0.0:9190->9090/tcp, [::]:9190->9090/tcp    proyecto-muebles-lab-api-gateway-1
291e43b22fb3        proyecto-muebles/frontend:lab           "/docker-entrypoint.s..." 9 days ago          Up 27 seconds      0.0.0.0:1100->80/tcp, [::]:1100->80/tcp      proyecto-muebles-lab-frontend-1
7241ff4c1b05        proyecto-muebles/authservice:lab        ".dotnet AuthService..." 10 days ago         Up 23 seconds      0.0.0.0:8181->8080/tcp, [::]:8181->8080/tcp    proyecto-muebles-lab-authservice-1
83166b93e048        proyecto-muebles/paymentervice:lab      ".dotnet PaymentServ..." 10 days ago         Up 24 seconds      0.0.0.0:8183->8080/tcp, [::]:8183->8080/tcp    proyecto-muebles-lab-paymentervice-1
4cad6627e3af        proyecto-muebles/orderservice:lab       ".dotnet OrderService..." 10 days ago         Up 23 seconds      0.0.0.0:8184->8080/tcp, [::]:8184->8080/tcp    proyecto-muebles-lab-orderservice-1
33e6b323e57        proyecto-muebles/cartservice:lab        ".dotnet CartService..." 10 days ago         Up 23 seconds      0.0.0.0:8182->8080/tcp, [::]:8182->8080/tcp    proyecto-muebles-lab-cartservice-1
2bb04e2ae79        proyecto-muebles/catalogservice:lab     ".dotnet CatalogServ..." 11 days ago         Up 27 seconds      0.0.0.0:8182->8080/tcp, [::]:8182->8080/tcp    proyecto-muebles-lab-catalogservice-1
c1f86c292472        proyecto-muebles/inventoryservice:lab   ".dotnet InventorySer..." 11 days ago         Up 23 seconds      0.0.0.0:8186->8080/tcp, [::]:8186->8080/tcp    proyecto-muebles-lab-inventoryservice-1
b247a592ecdf        postgres:16-alpine                    "docker-entrypoint.s..." 12 days ago         Up 27 seconds (healthy) 0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp    proyecto-muebles-lab-postgres-1
4dfc1e220c34        postgres:16                            "docker-entrypoint.s..." 3 weeks ago         Up 27 seconds (healthy) 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp    proyecto-muebles-postgres

ENVIRONMENT: RQ3N3P RING64 /C:/Proyecto/Proyecto-Muebles4-main (Laboratorio/docker/izacion)
$ docker ps -a
CONTAINER ID        IMAGE                                     COMMAND                  CREATED              STATUS              PORTS              NAMES
7d443b58f13        proyecto-muebles/api-gateway:lab        "/docker-entrypoint.s..." 9 days ago          Up 55 seconds      0.0.0.0:9190->9090/tcp, [::]:9190->9090/tcp    proyecto-muebles-lab-api-gateway-1
291e43b22fb3        proyecto-muebles/frontend:lab           "/docker-entrypoint.s..." 9 days ago          Up 55 seconds      0.0.0.0:1100->80/tcp, [::]:1100->80/tcp      proyecto-muebles-lab-frontend-1
7241ff4c1b05        proyecto-muebles/authservice:lab        ".dotnet AuthService..." 10 days ago         Up 51 seconds      0.0.0.0:8181->8080/tcp, [::]:8181->8080/tcp    proyecto-muebles-lab-authservice-1
83166b93e048        proyecto-muebles/paymentervice:lab      ".dotnet PaymentServ..." 10 days ago         Up 52 seconds      0.0.0.0:8183->8080/tcp, [::]:8183->8080/tcp    proyecto-muebles-lab-paymentervice-1
4cad6627e3af        proyecto-muebles/orderservice:lab       ".dotnet OrderService..." 10 days ago         Up 51 seconds      0.0.0.0:8184->8080/tcp, [::]:8184->8080/tcp    proyecto-muebles-lab-orderservice-1
33e6b323e57        proyecto-muebles/cartservice:lab        ".dotnet CartService..." 10 days ago         Up 51 seconds      0.0.0.0:8182->8080/tcp, [::]:8182->8080/tcp    proyecto-muebles-lab-cartservice-1
2bb04e2ae79        proyecto-muebles/catalogservice:lab     ".dotnet CatalogServ..." 11 days ago         Up 51 seconds      0.0.0.0:8182->8080/tcp, [::]:8182->8080/tcp    proyecto-muebles-lab-catalogservice-1
c1f86c292472        proyecto-muebles/inventoryservice:lab   ".dotnet InventorySer..." 11 days ago         Up 51 seconds      0.0.0.0:8186->8080/tcp, [::]:8186->8080/tcp    proyecto-muebles-lab-inventoryservice-1
b247a592ecdf        postgres:16-alpine                    "docker-entrypoint.s..." 12 days ago         Up 51 seconds (healthy) 0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp    proyecto-muebles-lab-postgres-1
4dfc1e220c34        postgres:16                            "docker-entrypoint.s..." 3 weeks ago         Exited (255) 13 days ago 0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp    proyecto-muebles-gateway
3a7af8f3a45f        node:10-alpine                        "docker-entrypoint.s..." 3 weeks ago         Exited (255) 13 days ago 0.0.0.0:8082->8082/tcp    authservice
095a3823444        mcr.microsoft.com/dotnet/sdk:8.0      "sh -c 'dotnet run -..." 3 weeks ago         Exited (255) 13 days ago 0.0.0.0:8086->8086/tcp    inventoryservice
aeaa8dc13967        mcr.microsoft.com/dotnet/sdk:8.0      "sh -c 'dotnet run -..." 3 weeks ago         Exited (255) 13 days ago 0.0.0.0:8082->8082/tcp    catalogservice
222f8f60801        mcr.microsoft.com/dotnet/sdk:8.0      "sh -c 'dotnet run -..." 3 weeks ago         Exited (255) 13 days ago 0.0.0.0:8083->8083/tcp    cartservice
6e223601403        mcr.microsoft.com/dotnet/sdk:8.0      "sh -c 'dotnet run -..." 3 weeks ago         Exited (255) 13 days ago 0.0.0.0:8084->8084/tcp    orderservice
7532ef8fe0c        mcr.microsoft.com/dotnet/sdk:8.0      "sh -c 'dotnet run -..." 3 weeks ago         Exited (255) 13 days ago 0.0.0.0:8085->8085/tcp    paymentervice
44807a274d1        mcr.microsoft.com/dotnet/sdk:8.0      "sh -c 'dotnet run -..." 3 weeks ago         Exited (255) 13 days ago 0.0.0.0:8088->8088/tcp    projectomuebles-postgres
4dfc1e220c34        postgres:16                            "docker-entrypoint.s..." 3 weeks ago         Up 55 seconds (healthy) 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp    proyecto-muebles-postgres

ENVIRONMENT: RQ3N3P RING64 /C:/Proyecto/Proyecto-Muebles4-main (Laboratorio/docker/izacion)
$ dotnet test backend/services/AuthService/AuthService.Tests/AuthService.Tests.csproj
Determinando los proyectos que se van a restaurar...
Todos los proyectos están actualizados para la restauración.
AuthService.Api -> C:\Proyecto\Proyecto-Muebles4-main\backend\services\AuthService\AuthService.Api\bin\Debug\net8.0\AuthService.Api.dll
AuthService.Tests -> C:\Proyecto\Proyecto-Muebles4-main\backend\services\AuthService\AuthService.Tests\bin\Debug\net8.0\AuthService.Tests.dll
Serie de pruebas para C:\Proyecto\Proyecto-Muebles4-main\backend\services\AuthService\AuthService.Tests\bin\Debug\net8.0\AuthService.Tests.dll (.NETCoreApp,Version=v8.0)
Versión 17.11.1 (c44) de VSTest
Iniciando la ejecución de pruebas, espere...
1 archivos de prueba en total coincidieron con el patrón especificado.
Correctas: - Con errores: 0, Superado: 9, Omitido: 0, Total: 9, Duración: 1 s - AuthService.Tests.dll (net8.0)

```

## CatalogService: fortalecimiento de reglas de validación

CatalogService ya contaba con validación de dominio en `Product.Validate()`: identificador obligatorio, nombre obligatorio, categoría obligatoria y precio mayor que cero. Por esta razón no se duplicaron reglas existentes; la mejora se concentró en agregar restricciones nuevas y sencillas sobre la longitud de los datos.

### 1. Mejoras implementadas

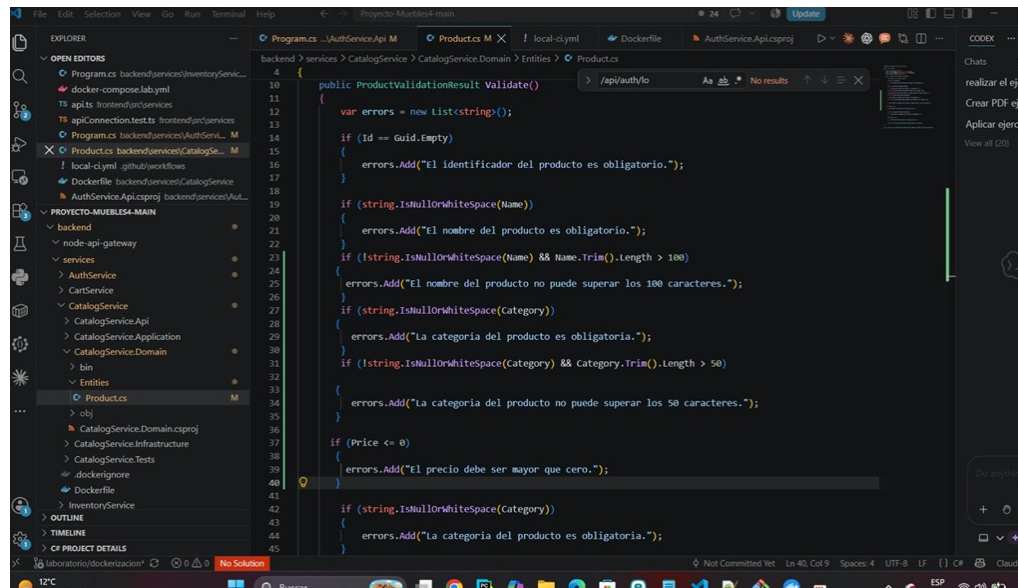
Nombre del producto: máximo 100 caracteres cuando el valor no está vacío.

Categoría del producto: máximo 50 caracteres cuando el valor no está vacío.

Se conservan las reglas existentes de identificador, obligatoriedad y precio positivo.

Durante la edición se eliminó una validación duplicada de precio, dejando una única regla equivalente.





## Beneficio y justificación

Los límites de longitud son una protección simple de calidad de datos. Evitan que nombres o categorías desproporcionadas lleguen al procesamiento del catálogo, reducen inconsistencias de presentación y hacen explícitas reglas de dominio que antes no estaban definidas. La validación se mantiene en la entidad Product, donde ya residían las reglas existentes, evitando crear lógica paralela.

Desde una perspectiva de ingeniería de software, las decisiones adoptadas en el proyecto se alinean con prácticas reconocidas para automatizar el ciclo de desarrollo, controlar versiones y fortalecer la trazabilidad de los artefactos. GitHub Actions permite automatizar flujos de trabajo de integración y despliegue dentro del repositorio, mientras que el uso de versionamiento semántico establece una convención explícita para comunicar cambios compatibles e

incompatibles. Asimismo, Docker distingue entre etiquetas y digest como mecanismos de identificación de imágenes, lo que resulta relevante para la reproducibilidad de los despliegues (Docker, 2026; GitHub, 2026; Preston-Werner, 2013).

## 8. Conclusiones

La implementación realizada permitió llevar el Proyecto Muebles desde un entorno principalmente local hacia una solución con mayor nivel de automatización, trazabilidad y reproducibilidad. La integración de GitHub Actions, Docker, GitHub Container Registry (GHCR) y Render permitió establecer un flujo continuo para validar, construir, publicar y desplegar los diferentes componentes de la aplicación.

Uno de los principales resultados fue la separación de la configuración del código mediante variables de entorno y la migración de la persistencia hacia PostgreSQL administrado en Render. Esta decisión facilita la diferenciación entre los ambientes de desarrollo y producción y evita mantener credenciales directamente dentro del repositorio. Además, la configuración del API Gateway y del frontend permitió adaptar la aplicación a las condiciones reales del entorno desplegado.

La automatización también permitió identificar problemas que no necesariamente eran visibles durante una validación únicamente local. El caso de CatalogService demostró la importancia de verificar el estado real de la aplicación después del despliegue. La incorporación de smoke tests posteriores al deploy fortalece el proceso de CD porque permite comprobar que producción no solamente recibió el despliegue, sino que realmente se encuentra disponible y funcional.

La estrategia de versionamiento semántico y la trazabilidad mediante SHA de los commits permiten relacionar el código fuente con los artefactos construidos y desplegados. Esto mejora la

capacidad de identificar qué versión se encuentra en ejecución y facilita futuras actividades de mantenimiento, recuperación o rollback.

Finalmente, el proyecto demuestra que la automatización de DevOps no se limita a ejecutar un pipeline de manera automática. Su valor está en establecer controles que permitan detectar errores, mantener la trazabilidad de los cambios, reducir actividades manuales y aumentar la confiabilidad de los despliegues. Para continuar aumentando la madurez de la solución, los siguientes pasos recomendados son mantener los smoke tests post-deploy, formalizar el proceso de releases mediante SemVer y consolidar la documentación técnica y el modelo de gestión de configuración.

### Referencias

- Docker. (2026). *Image digests. Docker Docs*. <https://docs.docker.com/dhi/explore/security-concepts/digests/>
- GitHub. (2026). *GitHub Actions documentation*. <https://docs.github.com/en/actions>
- National Institute of Standards and Technology. (2024). *Strategies for the integration of software supply chain security in DevSecOps CI/CD pipelines (NIST Special Publication 800-204D)*. <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-204D.pdf>
- PostgreSQL Global Development Group. (2026). *Environment variables. PostgreSQL documentation*. <https://www.postgresql.org/docs/18/libpq-envvars.html>
- Preston-Werner, T. (2013). *Semantic Versioning 2.0.0*. <https://semver.org/lang/es/>